Exam Planning System — Technical Documentation

Version: 1.0.0
Stack: Spring Boot 3.4 · Java 21 · PostgreSQL · Vanilla JS SPA
Port: 8081

Contents

Document	Description
[Architecture](architecture.md)	System layers, component diagram, request lifecycle
[Data Model](data-model.md)	Entity relationships, database schema, constraints
[API Reference](api-reference.md)	All REST endpoints with request/response shapes
[Planning Algorithm](planning-algorithm.md)	Core scheduling logic, conflict detection, auto-sche
[Security](security.md)	JWT authentication, Spring Security filter chain
[Configuration](configuration.md)	Environment variables, application properties
[Deployment](deployment.md)	Local setup, Docker Compose, production notes

---

Quick Start

```bash
# 1. Start PostgreSQL (port 5433)
docker-compose up -d db

# 2. Set environment
cp .env.example src/main/resources/application-local.properties
# fill in DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET

# 3. Run
./gradlew bootRun

# 4. Open
open http://localhost:8081
# Login: admin / admin123
```

Demo Data (seeded on first startup)

Entity	Count
Faculties	20
Departments	100
Instructors	134
Courses	200
Classrooms	30
Students	10,200
Exams	40

Architecture

Overview

The system follows a standard layered architecture: a Spring Boot backend exposes a REST API consume
gle-page application served from the same origin.

```
Browser (Vanilla JS SPA)
        ■  HTTP/JSON  (JWT Bearer token on every request)
        ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■  Spring Boot 3.4  (port 8081)                    ■
■                                                  ■
■  ■■■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■ ■
■  ■ Controllers ■→ ■   Services   ■→ ■  Repos   ■ ■
■  ■■■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■■■■■■  ■■■■■■■■■■■■■■■ ■
■         ↑                                ■     ■
■   JwtAuthFilter                          ▼     ■
■   SecurityConfig                  PostgreSQL   ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Layers

Controllers (`controller/`)
Thin HTTP adapters. Each controller maps to one domain resource and delegates all business logic to
Validation is handled by Bean Validation (`@Valid`) at the controller boundary.

Controller	Base Path	Purpose
`AuthController`	`/api/auth`	Login, logout, register
`FacultyController`	`/api/admin/faculties`	Faculty CRUD
`DepartmentController`	`/api/admin/departments`	Department CRUD
`InstructorController`	`/api/admin/instructors`	Instructor CRUD, CSV import
`CourseController`	`/api/admin/courses`	Course CRUD
`ClassroomController`	`/api/admin/classrooms`	Classroom CRUD
`StudentController`	`/api/admin/students`	Student CRUD, CSV import
`ExamController`	`/api/admin/exams`	Exam CRUD, Excel export
`ExamPlanningController`	`/api/admin/exam-planning`	Planning algorithm, auto-schedule, conflic
`ExamAssignmentController`	`/api/admin/exam-assignments`	View/manage student seat assignments
`InvigilatorAssignmentController`	`/api/admin/invigilator-assignments`	View/manage invigilator ts
`QueryController`	`/api/admin/query`	Cross-entity search and analytics
`UserController`	`/api/admin/users`	User management

Services (`service/`)
All business logic lives here. Services are `@Transactional` where they write to the database. The m
x service is `ExamPlanningService` — see [Planning Algorithm](planning-algorithm.md).

Repositories (`repository/`)

Spring Data JPA interfaces extending `JpaRepository`. Custom JPQL queries are annotated with `@Query
on the interface methods — no XML mapping files.

DTOs (`dto/`)

Request and response objects are separate from JPA entities. Controllers accept `*CreateRequest` obj
eturn `*Response` objects, preventing accidental field exposure and decoupling the API contract fr
ema.

Entities (`entity/`)

JPA-mapped POJOs. Relationships are declared with `@ManyToOne` / `@OneToMany`. Unique constraints ar
at the DB level via `@UniqueConstraint` on the `@Table` annotation.

Request Lifecycle

```
1. Request arrives at JwtAuthFilter
2. Filter extracts Bearer token from Authorization header
3. JwtService validates signature + expiry + blacklist check
4. Authentication set in SecurityContextHolder
5. Spring Security checks endpoint authorization rules
6. Request dispatched to matching @RestController method
7. @Valid triggers Bean Validation on request body
8. Controller calls Service
9. Service interacts with Repositories (JPA → HikariCP → PostgreSQL)
10. Service returns domain object / throws domain exception
11. GlobalExceptionHandler converts exceptions to structured JSON error responses
12. Controller wraps result in ResponseEntity and returns
```

Frontend (SPA)

The frontend is a hand-written Vanilla JS SPA with hash-based routing (`window.location.hash`). Ther
rameworks or build tools — files are served as-is by Spring Boot's static resource handler from `s
sources/static/`.

```
static/
■■■ index.html          — single HTML shell, loads router
■■■ css/                — scoped component styles
■■■ js/
    ■■■ api.js          — fetch wrapper, attaches JWT, handles 401
    ■■■ router.js       — hash-based routing, view lifecycle (mount/unmount)
    ■■■ auth.js         — login state, token storage in localStorage
    ■■■ components/     — Navbar, CrudView, Pagination, Modal
    ■■■ views/          — one file per route (DashboardView, StudentsView, …)
```

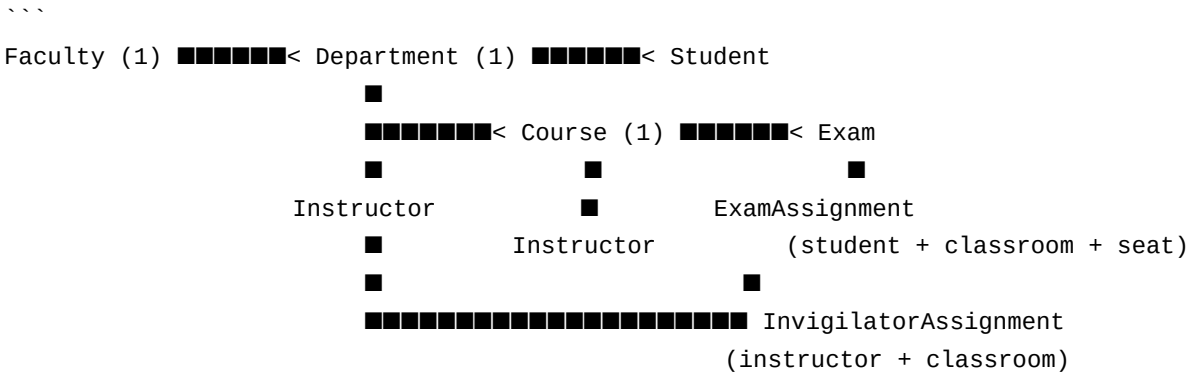
Each view exports a class with `getHtml()` (returns HTML string) and `mount()` (attaches event liste
hes data). The router calls `unmount()` on the outgoing view before rendering the next one.

Technology Choices

Concern	Choice	Reason
ORM	Spring Data JPA / Hibernate	Standard for Spring Boot; JPQL queries are portable
Connection pool	HikariCP	Default in Spring Boot; tuned via `application.properties`
Security	Spring Security 6 + JJWT	Stateless JWT; token blacklist for logout support
API docs	springdoc-openapi (Swagger UI)	Auto-generated from annotations; available at `/swagge
Excel export	Apache POI	`ExamController` generates `.xlsx` exam schedules
CSV import	OpenCSV	Student and instructor bulk upload
Passwords	BCryptPasswordEncoder	Strength factor default (10 rounds)

Data Model

Entity Relationship Diagram



Tables

`faculties`

Column	Type	Constraints
`faculty_id`	BIGINT	PK, auto-increment
`faculty_name`	VARCHAR(150)	NOT NULL, UNIQUE

`departments`

Column	Type	Constraints
`department_id`	BIGINT	PK, auto-increment
`department_name`	VARCHAR(150)	NOT NULL
`faculty_id`	BIGINT	FK → faculties

`instructors`

Column	Type	Constraints
`instructor_id`	BIGINT	PK, auto-increment
`staff_no`	VARCHAR(50)	NOT NULL, UNIQUE
`full_name`	VARCHAR(150)	NOT NULL
`email`	VARCHAR(150)	NOT NULL, UNIQUE
`department_id`	BIGINT	FK → departments
`user_id`	BIGINT	FK → users (nullable)
`is_available_for_invigilation`	BOOLEAN	NOT NULL, default true
`duty_count`	INTEGER	NOT NULL, default 0

`duty_count` is incremented on each invigilator assignment and decremented on plan reset. The planni
  hm sorts by `duty_count ASC` to distribute workload fairly.

`courses`

Column	Type	Constraints
`course_id`	BIGINT	PK, auto-increment
`course_code`	VARCHAR(20)	NOT NULL, UNIQUE
`course_name`	VARCHAR(200)	NOT NULL
`semester`	VARCHAR(50)	
`credit_hours`	INTEGER	
`instructor_id`	BIGINT	FK → instructors
`department_id`	BIGINT	FK → departments

`classrooms`

Column	Type	Constraints
`classroom_id`	BIGINT	PK, auto-increment
`campus`	VARCHAR(100)	NOT NULL
`building`	VARCHAR(100)	NOT NULL
`room_name`	VARCHAR(50)	NOT NULL, UNIQUE
`capacity`	INTEGER	NOT NULL
`is_available`	BOOLEAN	NOT NULL, default true
`technical_features`	VARCHAR(500)	

Classrooms with `is_available = false` are excluded from planning entirely.

`students`

Column	Type	Constraints
`student_id`	BIGINT	PK, auto-increment
`student_no`	VARCHAR(50)	NOT NULL, UNIQUE
`tc_no`	VARCHAR(20)	NOT NULL, UNIQUE
`full_name`	VARCHAR(150)	NOT NULL
`faculty_id`	BIGINT	FK → faculties
`department_id`	BIGINT	FK → departments
`user_id`	BIGINT	FK → users (nullable)

`exams`

Column	Type	Constraints
`exam_id`	BIGINT	PK, auto-increment
`exam_name`	VARCHAR(150)	NOT NULL
`exam_type`	VARCHAR(50)	e.g. MIDTERM, FINAL
`exam_date`	DATE	NOT NULL
`exam_time`	TIME	NOT NULL
`duration`	INTEGER	minutes
`course_id`	BIGINT	FK → courses, NOT NULL
`classroom_id`	BIGINT	FK → classrooms (nullable — set at planning)
`is_common_exam`	BOOLEAN	NOT NULL, default false
`created_at`	TIMESTAMP	NOT NULL, set on insert

`exam_assignments`

Junction table: one row per student per exam, carries the physical seat.

Column	Type	Constraints
`assignment_id`	BIGINT	PK, auto-increment
`exam_id`	BIGINT	FK → exams, NOT NULL
`student_id`	BIGINT	FK → students, NOT NULL
`classroom_id`	BIGINT	FK → classrooms, NOT NULL
`seat_number`	INTEGER	sequential within classroom
`created_at`	TIMESTAMP	NOT NULL, set on insert

Unique constraint: `(exam_id, student_id)` — a student can only be assigned to a given exam once

`invigilator_assignments`

Junction table: one row per instructor per exam room.

Column	Type	Constraints
`invigilation_id`	BIGINT	PK, auto-increment
`exam_id`	BIGINT	FK → exams, NOT NULL
`instructor_id`	BIGINT	FK → instructors, NOT NULL
`classroom_id`	BIGINT	FK → classrooms, NOT NULL
`created_at`	TIMESTAMP	NOT NULL, set on insert

Unique constraint: `(exam_id, instructor_id)` — an instructor can only be assigned to a given ex
cross all rooms).

`users`

Column	Type	Constraints
`user_id`	BIGINT	PK, auto-increment
`username`	VARCHAR(50)	NOT NULL, UNIQUE
`password_hash`	VARCHAR(255)	NOT NULL (BCrypt)
`role`	VARCHAR(20)	NOT NULL — enum: ADMIN, INSTRUCTOR, STUDENT

`blacklisted_tokens`

Column	Type	Constraints
`id`	BIGINT	PK, auto-increment
`token`	TEXT	NOT NULL
`blacklisted_at`	TIMESTAMP	NOT NULL

Tokens are added here on logout. Every authenticated request checks this table to prevent reuse of v
ogged-out JWTs.

Key Invariants

- A student cannot be in `exam_assignments` twice for the same exam (`UNIQUE(exam_id, student_id)`).
- A student cannot have two assignments at the same `(exam_date, exam_time)` — enforced in the servi
efore writing.
- An instructor cannot invigilate two exams at the same `(exam_date, exam_time)` — enforced in the s
er.
- A classroom cannot host two exams at the same `(exam_date, exam_time)` — enforced by filtering occ

srooms before assignment.
- Deleting a plan (reset) decrements `instructor.duty_count` for every removed invigilator assignmen

API Reference

All endpoints (except `/api/auth/login`) require a `Authorization: Bearer <token>` header.

Interactive documentation is available at **`http://localhost:8081/swagger-ui.html`**.

Authentication — `/api/auth`

POST `/api/auth/login`
Authenticate and receive a JWT.

Request
```json
{ "username": "admin", "password": "admin123" }
```
Response 200
```json
{ "token": "<jwt>", "username": "admin", "role": "ADMIN" }
```

POST `/api/auth/logout`
Blacklists the current token. Requires `Authorization` header.

Response 200 `{ "message": "Logged out successfully" }`

POST `/api/auth/register`
Create a new user account.

Request
```json
{ "username": "instructor1", "password": "pass123", "role": "INSTRUCTOR" }
```

Exam Planning — `/api/admin/exam-planning`

POST `/api/admin/exam-planning/plan/{examId}`
Runs the planning algorithm for an exam.

Path param: `examId` — the exam to plan
Query param: `dryRun=true` — simulate without writing to DB
Body: Array of student IDs
```json
[1, 2, 3, 4, 5]
```

Response 200
```json
```

```
{
  "examId": 1,
  "examName": "Introduction to Computer Engineering Midterm",
  "examDate": "2026-07-01",
  "examTime": "09:00:00",
  "totalStudents": 5,
  "classroomsUsed": 1,
  "invigilatorsAssigned": 1,
  "dryRun": false,
  "classrooms": [
    {
      "classroom": "Main - Block A - A-101",
      "capacity": 50,
      "studentsAssigned": 5,
      "invigilatorsAssigned": 1,
      "invigilatorRule": "1–50 students → 1 invigilator",
      "studentNumbers": ["STU-0000001", "STU-0000002"],
      "invigilatorNames": ["Dr. Ali Aliyev (duties: 1)"]
    }
  ]
}
```

Error responses

Code	Condition
400	Student already assigned to this exam
400	Student has a scheduling conflict at this date/time
400	Insufficient classroom capacity
400	Not enough available invigilators
404	Exam or student not found

DELETE `/api/admin/exam-planning/plan/{examId}`

Removes all student and invigilator assignments for an exam and decrements instructor duty counts.

Response 200

```json
{
  "examId": 1,
  "examName": "...",
  "studentAssignmentsCleared": 120,
  "invigilatorAssignmentsCleared": 3,
  "message": "Plan was reset successfully"
}
```

POST `/api/admin/exam-planning/validate/{examId}`

Check which students can be included in planning before committing.

Body: Array of student IDs

Response 200

```json
{
  "examId": 1,
  "conflicts": [
    {
      "studentId": 5,
      "studentNo": "STU-0000005",
      "studentName": "Ali Aliyev",
      "reason": "Already assigned to this exam"
    }
  ],
  "validStudentIds": [1, 2, 3, 4],
  "conflictCount": 1,
  "validCount": 4
}
```

POST `/api/admin/exam-planning/auto-schedule`
Finds the first viable date+time slot and creates + plans the exam automatically.

Request
```json
{
  "courseId": 1,
  "studentIds": [1, 2, 3],
  "examName": "Optional custom name",
  "examType": "MIDTERM",
  "duration": 90,
  "preferredDate": "2026-07-01"
}
```
All fields except `courseId` and `studentIds` are optional.

Response 200 — same shape as `POST /plan/{examId}` with `"autoScheduled": true` added.

GET `/api/admin/exam-planning/conflicts`
Returns all scheduling conflicts across every exam in the system.

GET `/api/admin/exam-planning/conflicts/{examId}`
Returns conflicts scoped to a single exam.

Conflict types: `STUDENT_DOUBLE_BOOKED`, `INSTRUCTOR_DOUBLE_BOOKED`, `CLASSROOM_DOUBLE_BOOKED`

Exams — `/api/admin/exams`

Method	Path	Description
GET	`/api/admin/exams`	Paginated list. Params: `page`, `size`, `sort`, `search`, `type`, `date
GET	`/api/admin/exams/{id}`	Single exam

POST	`/api/admin/exams`	Create exam
PUT	`/api/admin/exams/{id}`	Update exam
DELETE	`/api/admin/exams/{id}`	Delete exam
GET	`/api/admin/exams/{id}/export`	Download `.xlsx` schedule

Create/Update request
```json
{
  "examName": "Advanced Topics in CS Final",
  "examType": "FINAL",
  "examDate": "2026-07-21",
  "examTime": "09:00:00",
  "duration": 120,
  "courseId": 1,
  "isCommonExam": false
}
```

Students — `/api/admin/students`

Method	Path	Description
GET	`/api/admin/students`	Paginated. Params: `page`, `size`, `search`, `departmentId`, `facult
GET	`/api/admin/students/{id}`	Single student
POST	`/api/admin/students`	Create student
PUT	`/api/admin/students/{id}`	Update student
DELETE	`/api/admin/students/{id}`	Delete student
POST	`/api/admin/students/import`	CSV bulk import (`multipart/form-data`, field `file`)

Instructors — `/api/admin/instructors`

Method	Path	Description
GET	`/api/admin/instructors`	Paginated. Params: `page`, `size`, `search`, `departmentId`
GET	`/api/admin/instructors/{id}`	Single instructor
POST	`/api/admin/instructors`	Create instructor
PUT	`/api/admin/instructors/{id}`	Update instructor
DELETE	`/api/admin/instructors/{id}`	Delete instructor
POST	`/api/admin/instructors/import`	CSV bulk import

Courses — `/api/admin/courses`

Method	Path	Description
GET	`/api/admin/courses`	Paginated. Params: `page`, `size`, `search`, `departmentId`

GET	`/api/admin/courses/{id}`	Single course
POST	`/api/admin/courses`	Create course
PUT	`/api/admin/courses/{id}`	Update course
DELETE	`/api/admin/courses/{id}`	Delete course

Classrooms — `/api/admin/classrooms`

Method	Path	Description
GET	`/api/admin/classrooms`	Paginated. Params: `page`, `size`, `search`, `campus`
GET	`/api/admin/classrooms/{id}`	Single classroom
POST	`/api/admin/classrooms`	Create classroom
PUT	`/api/admin/classrooms/{id}`	Update classroom
DELETE	`/api/admin/classrooms/{id}`	Delete classroom

Faculties & Departments

Method	Path	Description
GET	`/api/admin/faculties`	All faculties (paginated)
POST	`/api/admin/faculties`	Create faculty
PUT	`/api/admin/faculties/{id}`	Update faculty
DELETE	`/api/admin/faculties/{id}`	Delete faculty
GET	`/api/admin/departments`	All departments (paginated)
POST	`/api/admin/departments`	Create department
PUT	`/api/admin/departments/{id}`	Update department
DELETE	`/api/admin/departments/{id}`	Delete department

Exam Assignments — `/api/admin/exam-assignments`

Method	Path	Description
GET	`/api/admin/exam-assignments`	All assignments (paginated, filter by `examId`)
GET	`/api/admin/exam-assignments/{id}`	Single assignment
DELETE	`/api/admin/exam-assignments/{id}`	Remove one student from an exam

Invigilator Assignments — `/api/admin/invigilator-assignments`

Method	Path	Description
GET	`/api/admin/invigilator-assignments`	All invigilator assignments (filter by `examId`)
DELETE	`/api/admin/invigilator-assignments/{id}`	Remove one invigilator assignment

Query / Analytics — `/api/admin/query`

Method	Path	Description
GET	`/api/admin/query/students`	Search students by name, TC no, student no
GET	`/api/admin/query/exams-by-date`	Exams on a given date
GET	`/api/admin/query/student-exam-history/{studentId}`	All exams a student is assigned to
GET	`/api/admin/query/instructor-workload`	Duty count ranking across all instructors
GET	`/api/admin/query/classroom-utilization`	Utilization stats per classroom

Pagination

All list endpoints return a standard page envelope:

```json
{
  "content": [ ... ],
  "totalElements": 10200,
  "totalPages": 204,
  "size": 50,
  "number": 0
}
```

Error Responses

All errors are returned as JSON:

```json
{
  "status": 404,
  "error": "Not Found",
  "message": "Exam not found with id: 99",
  "timestamp": "2026-06-10T14:30:00"
}
```

HTTP Status	Condition
400	Validation failure, business rule violation, insufficient capacity
401	Missing or invalid JWT
403	Authenticated but insufficient role
404	Requested resource does not exist
409	Duplicate resource (unique constraint)
500	Unexpected server error

Planning Algorithm

The core of the system. Implemented in `ExamPlanningService`.

Manual Planning — `planExam(examId, studentIds, dryRun)`

Takes an exam and a list of student IDs, distributes students across classrooms, assigns invigilator
tes the result to the database.

Step-by-step

```
1. LOAD
   ■■■ Fetch exam by ID
   ■■■ Fetch all students by IDs (validated)

2. PRE-CONFLICT CHECK (fail fast, no writes)
   ■■■ Are any students already assigned to this exam?
   ■   → DuplicateResourceException
   ■■■ Do any students have another exam at (date, time)?
       → DuplicateResourceException

3. FIND FREE CLASSROOMS
   ■■■ Query all exams at (date, time) that have a classroom assigned
   ■■■ Filter those classroom IDs out of the available set
   ■■■ Keep only classrooms where is_available = true
   ■■■ Sort DESC by capacity (largest rooms first)

4. CAPACITY CHECK (fail fast)
   ■■■ totalCapacity = sum of all available classroom capacities
   ■■■ totalCapacity < students.size → InsufficientCapacityException

5. FIND FREE INSTRUCTORS
   ■■■ Already on this exam (invigilator_assignments.exam_id = examId)
   ■■■ Busy at (date, time) on another exam
   ■■■ is_available_for_invigilation = false
   ■■■ Sort ASC by duty_count (fairness)

6. PRE-VALIDATE INVIGILATORS NEEDED (fail fast, no writes)
   ■■■ Simulate room fill using greedy algorithm
   ■■■ Sum invigilators needed across all rooms (apply invigilator rule)
   ■■■ needed > available.size → InsufficientCapacityException with clear message

7. ASSIGN (greedy, left-to-right)
   ■■■ Sort students by student_no (deterministic)
   ■■■ For each classroom (largest → smallest):
   ■   ■■■ Fill seats 1…capacity sequentially with next students
   ■   ■■■ Count students placed → apply invigilator rule
   ■   ■■■ Take N instructors from available list (lowest duty first)
```

```
   ■■■ Stop when all students placed

8. PERSIST (unless dryRun=true)
   ■■■ saveAll(ExamAssignment[])
   ■■■ saveAll(InvigilatorAssignment[])
   ■■■ saveAll(updated Instructors) — duty_count incremented

9. RETURN summary map with classroom breakdown
```

Invigilator Rule

Students in room	Invigilators required
1 – 50	1
51 – 100	2
101 +	3

Dry Run Mode

When `dryRun=true`, steps 1–7 execute normally (all validation runs) but step 8 is skipped. The resp
des the full room breakdown so the user can preview the plan before committing.

Validate Before Planning — `validateStudentsForPlan(examId, studentIds)`

A lighter check that returns which students have conflicts and which are eligible, without running t
gorithm. Use this to filter the student list in the UI before calling `planExam`.

Returns:
- `conflicts[]` — students with `reason` (`Already assigned` or `Scheduling conflict`)
- `validStudentIds[]` — IDs safe to include in planning

Reset Plan — `resetExamPlan(examId)`

Reverses a plan completely:

1. Load all `ExamAssignment` rows for the exam
2. Load all `InvigilatorAssignment` rows for the exam
3. For each invigilator assignment: `instructor.dutyCount -= 1`
4. Save updated instructors
5. Delete all exam assignments
6. Delete all invigilator assignments

The duty count decrement ensures the fairness counter stays accurate when plans are reworked.

Auto-Schedule — `autoScheduleExam(request)`

Finds the first viable (date, time) slot within 30 days, then creates the exam and runs `planExam`.

```
Input: courseId, studentIds, optional preferredDate, examType, duration, examName

1. Resolve course → fail if not found
2. Resolve students → fail if any not found
3. startDate = preferredDate ?? tomorrow
4. For dayOffset in 0..29:
   For timeSlot in [09:00, 11:00, 13:00, 15:00]:
     a. isSlotViable? — no student has any assignment at (date, slot)
     b. availableClassrooms at (date, slot) — enough total capacity?
     c. availableInstructors at (date, slot) — enough for the room fill?
     d. All pass → create Exam, call planExam, return result with autoScheduled=true
5. No slot found in 30 days → InsufficientCapacityException
```

The slot search is purely database-driven — no in-memory state is maintained between iterations.

Conflict Detection — `detectAllConflicts()` / `detectConflicts(examId)`

Three conflict types are detected:

STUDENT_DOUBLE_BOOKED
A student appears in `exam_assignments` for two different exams at the same `(exam_date, exam_time)`

Detected via:
`ExamAssignmentRepository.findConflictingStudentAssignments()` — a JPQL query that self-joins `exam_
  s` on `student_id` where `exam_date` and `exam_time` match but `exam_id` differs.

INSTRUCTOR_DOUBLE_BOOKED
An instructor appears in `invigilator_assignments` for two different exams at the same `(exam_date,
  `.

Detected via:
`InvigilatorAssignmentRepository.findConflictingInvigilatorAssignments()`

CLASSROOM_DOUBLE_BOOKED
A classroom hosts students from two different exams at the same `(exam_date, exam_time)`.

Detected via:
`ExamAssignmentRepository.findConflictingClassroomAssignments()`

All three queries are implemented as `@Query` JPQL on the repository interfaces.

Fairness Distribution

Instructor workload is tracked via `instructor.duty_count`. Every invocation of `planExam` increment
  nts for assigned invigilators; every `resetExamPlan` decrements them. The planning algorithm alway
  ailable instructors `ASC by duty_count`, so the instructor with the fewest duties is always picked
  is produces an approximately even distribution over time without requiring a separate scheduling p

Security

Authentication Model

The system is stateless. There are no server-side sessions. Every request must carry a signed JWT in
  orization: Bearer <token>` header. The server validates the token on every request via a servlet f

JWT

Library: `io.jsonwebtoken` (JJWT)
Algorithm: HMAC-SHA256 (HS256)
Expiry: 24 hours (`jwt.expiration=86400000` ms)
Secret: Injected via `JWT_SECRET` env var (minimum 32 characters)

The token payload contains:
- `sub` — username
- Standard `iat` (issued at) and `exp` (expiration) claims

The secret must be at least 256 bits for HS256. A suitable value can be generated with:
```bash
openssl rand -base64 48
```

Filter Chain

`JwtAuthFilter extends OncePerRequestFilter` runs on every request before Spring Security's authoriz
  ks.

```
Incoming request
      ■
      ▼
JwtAuthFilter.doFilterInternal()
      ■■■ Extract "Authorization: Bearer ..." header
      ■■■ No header → pass through (Spring Security will reject protected routes)
      ■■■ Extract username from token (JwtService.extractUsername)
      ■■■ No existing authentication in SecurityContext →
      ■     ■■■ Load UserDetails from DB (UserDetailsServiceImpl)
      ■     ■■■ JwtService.isTokenValid(token, userDetails)
      ■     ■     ■■■ Verify signature
      ■     ■     ■■■ Check expiry
      ■     ■     ■■■ Check token not in blacklisted_tokens table
      ■     ■■■ Set UsernamePasswordAuthenticationToken in SecurityContextHolder
      ■■■ Pass to next filter
```

Security Configuration — `SecurityConfig`

```
Public (no token required):
  POST /api/auth/login
```

```
  GET  /swagger-ui/**
  GET  /v3/api-docs/**
  GET  /swagger-custom.css
  GET  /  (SPA shell)
  GET  /index.html
  GET  /css/**
  GET  /js/**

Protected:
  All other /api/** → requires valid JWT
```

CSRF protection is disabled (stateless JWT API — no cookies).
CORS is permissive in development. Restrict `allowedOrigins` in production.

Logout & Token Blacklisting

On `POST /api/auth/logout`:
1. Extract the token from the `Authorization` header
2. Insert it into `blacklisted_tokens` with the current timestamp
3. Subsequent requests carrying that token will be rejected even if not expired

The blacklist is a PostgreSQL table — no TTL cleanup is currently implemented. In production, expire
  hould be purged periodically (tokens older than `jwt.expiration` cannot be valid regardless).

Password Hashing

`BCryptPasswordEncoder` with default strength (10 rounds). Passwords are never stored in plaintext o

Roles

Role	Access
`ADMIN`	Full access to all `/api/admin/**` endpoints
`INSTRUCTOR`	Intended for instructor-scoped views (not yet enforced at endpoint level)
`STUDENT`	Intended for student portal (not yet implemented)

Currently all `/api/admin/**` routes require only authentication, not a specific role. Role-based re
  at the method level can be added with `@PreAuthorize("hasRole('ADMIN')")`.

Security Hardening Notes

- The default admin password (`admin123`) must be changed immediately after first login in any non-d
  nment.
- `JWT_SECRET` must never be committed to source control — use environment variables or a secrets ma
- `application-local.properties` is listed in `.gitignore` and must not be committed.
- SQL injection is not possible — all queries go through JPA/JPQL with parameterized binding.
- The API returns generic error messages for auth failures (401/403) without revealing internal stat

Configuration

Environment Variables

All sensitive values are injected via environment variables. The application reads them from `applic
  l.properties` when running with the `local` Spring profile (default).

Variable	Required	Description	Example
`DB_URL`	Yes	Full JDBC URL to PostgreSQL	`jdbc:postgresql://localhost:5433/exam_planning_sys
`DB_USERNAME`	Yes	Database username	`exam_user`
`DB_PASSWORD`	Yes	Database password	`exam_password`
`JWT_SECRET`	Yes	HMAC-SHA256 signing key (min 32 chars)	`openssl rand -base64 48`
`SPRING_PROFILES_ACTIVE`	No	Override active profile	`prod`

Setting up locally

```bash
cp .env.example src/main/resources/application-local.properties
# Edit the file and fill in real values
```

`application-local.properties` is in `.gitignore` — never commit it.

Application Properties

Key settings in `src/main/resources/application.properties`:

Server
```properties
server.port=8081
spring.profiles.active=${SPRING_PROFILES_ACTIVE:local}
```

Database
```properties
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

`ddl-auto=update` — Hibernate applies schema changes on startup. Suitable for development. For produ
  tch to `validate` and manage migrations with Flyway or Liquibase.

HikariCP Connection Pool
```properties
spring.datasource.hikari.maximum-pool-size=10
```

```
spring.datasource.hikari.minimum-idle=2
spring.datasource.hikari.connection-timeout=20000
spring.datasource.hikari.idle-timeout=300000
spring.datasource.hikari.max-lifetime=1200000
```

Hibernate Batch Inserts

```properties
spring.jpa.properties.hibernate.jdbc.batch_size=100
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

Batch inserts are used by `DataInitializer` to seed 10,200 students efficiently. The batch size of 1
   applied to planning operations (`saveAll` on bulk assignment lists).

JWT

```properties
jwt.secret=${JWT_SECRET}
jwt.expiration=86400000   # 24 hours in milliseconds
```

Swagger / OpenAPI

```properties
springdoc.api-docs.path=/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.swagger-ui.operationsSorter=alpha
springdoc.swagger-ui.tagsSorter=alpha
springdoc.swagger-ui.docExpansion=none
springdoc.swagger-ui.filter=true
```

Logging

```properties
spring.jpa.show-sql=false
logging.level.org.hibernate.SQL=WARN
logging.level.org.hibernate.orm.jdbc.bind=WARN
```

SQL logging is disabled by default to avoid noise. To enable during debugging:

```properties
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.SQL=DEBUG
```

Spring Profiles

| Profile | Properties loaded | Use case |

```
|---|---|---|
| `local` (default) | `application.properties` + `application-local.properties` | Local development
| `prod` | `application.properties` + `application-prod.properties` | Production (create this file)

Switch profile:
```bash
SPRING_PROFILES_ACTIVE=prod ./gradlew bootRun
or
java -jar app.jar --spring.profiles.active=prod
```

---

build.gradle — Key Dependencies

Dependency	Purpose
`spring-boot-starter-web`	REST controllers, embedded Tomcat
`spring-boot-starter-data-jpa`	JPA / Hibernate ORM
`spring-boot-starter-security`	Authentication/authorization
`spring-boot-starter-validation`	Bean Validation (`@Valid`, `@NotNull`, etc.)
`postgresql`	JDBC driver
`jjwt-api` / `jjwt-impl`	JWT generation and validation
`springdoc-openapi-starter-webmvc-ui`	Swagger UI
`poi-ooxml`	Apache POI — Excel export
`opencsv`	CSV import
`lombok`	Boilerplate reduction (`@Data`, `@Builder`, etc.)
```

Deployment

Prerequisites

Tool	Version
Java	21 +
Gradle	Wrapper included (`./gradlew`)
PostgreSQL	14 + (or Docker)

Local Development

1. Start PostgreSQL

With Docker (recommended):
```bash
docker-compose up -d db
```
This starts PostgreSQL on port **5433** with:
- Database: `exam_planning_system`
- User: `exam_user`
- Password: `exam_password`

Without Docker: create the database and user manually:
```sql
CREATE USER exam_user WITH PASSWORD 'exam_password';
CREATE DATABASE exam_planning_system OWNER exam_user;
```

2. Configure

```bash
cp .env.example src/main/resources/application-local.properties
```

The file should contain:
```properties
DB_URL=jdbc:postgresql://localhost:5433/exam_planning_system
DB_USERNAME=exam_user
DB_PASSWORD=exam_password
JWT_SECRET=<at-least-32-characters>
```

3. Run

```bash
./gradlew bootRun
```

The application starts on **http://localhost:8081**.

4. First startup

On first run, `DataInitializer` seeds the database:
- 20 faculties, 100 departments
- 134 instructors, 200 courses
- 30 classrooms, **10,200 students**, 40 exams
- Default admin: `admin` / `admin123`

Seeding is skipped on subsequent starts (guarded by `facultyRepository.count() == 0`).

Docker Compose

`docker-compose.yml` defines two services:

```yaml
services:
  db:
    image: postgres:16-alpine
    ports: ["5433:5432"]
    environment:
      POSTGRES_DB: exam_planning_system
      POSTGRES_USER: exam_user
      POSTGRES_PASSWORD: exam_password
    volumes:
      - pgdata:/var/lib/postgresql/data

  app:
    build: .
    ports: ["8081:8081"]
    depends_on: [db]
    environment:
      DB_URL: jdbc:postgresql://db:5432/exam_planning_system
      DB_USERNAME: exam_user
      DB_PASSWORD: exam_password
      JWT_SECRET: ${JWT_SECRET}
```

Run full stack:
```bash
JWT_SECRET=your-secret ./docker-compose up --build
```

Run only the database (app runs locally):
```bash
docker-compose up -d db
```

Building a JAR

```bash
./gradlew bootJar
# Output: build/libs/exam-planning-system-*.jar
```

Run the JAR:
```bash
java -jar build/libs/exam-planning-system-*.jar \
  --DB_URL=jdbc:postgresql://localhost:5433/exam_planning_system \
  --DB_USERNAME=exam_user \
  --DB_PASSWORD=exam_password \
  --JWT_SECRET=your-secret
```

Production Checklist

- [ ] Change default admin password immediately after first login
- [ ] Set a strong, unique `JWT_SECRET` (min 32 chars, ideally 48+ from `openssl rand -base64 48`)
- [ ] Switch `spring.jpa.hibernate.ddl-auto` from `update` to `validate`; manage schema with Flyway
  se
- [ ] Restrict CORS `allowedOrigins` in `SecurityConfig` to your actual frontend domain
- [ ] Put the app behind a reverse proxy (nginx / Caddy) with HTTPS
- [ ] Set up log aggregation (the app writes to stdout — capture with your platform's log driver)
- [ ] Schedule a periodic cleanup job for `blacklisted_tokens` older than 24 hours
- [ ] Set up database backups for `pgdata` volume

Useful URLs

URL	Description
`http://localhost:8081`	Application UI
`http://localhost:8081/swagger-ui.html`	Interactive API documentation
`http://localhost:8081/v3/api-docs`	Raw OpenAPI JSON